MODUL PRAKTIKUM MINGGU 06

KONEKTIVITAS BASIS DATA PERSISTEN (DATABASE CONNECTIVITY), ARSITEKTUR PDO & PARAMETER BINDING, FLUENT QUERY BUILDER, TRANSAKSI BASIS DATA ACID (DB::TRANSACTION & LOCKFORUPDATE), SERTA MANAJEMEN SESI & AUTENTIKASI PADA LARAVEL 11.X

Mata Kuliah	: Pemrograman & Pengujian Web (12S3101)	**Dosen Pengampu**	: Chandro Pardede, S.Kom., M.Sc.
Keterkaitan Materi	: Lanjutan MVC Week 5 ke DB ACID & Session	**Alokasi Waktu**	: 170' Lab + 360' Mandiri
Tahun Akademik	: Semester Ganjil 2026/2027	**Format Tugas**	: GitHub (ppw-2026-week6-[NIM])

I. Capaian Pembelajaran & Tujuan Praktikum

> **Sub-CPMK (SubCPMK12-02-09-05):** Mahasiswa mampu berinteraksi dengan basis data sebagai persistence storage serta mampu memanajemen sesi pengguna aplikasi web dengan tepat. [C3, A2, P2]

1.2 Indikator Ketercapaian Terukur

- **Arsitektur Konektivitas Database & PDO:** Menjelaskan perbedaan memori efemeral (RAM) dengan persistence storage (RDBMS), siklus hidup koneksi TCP/IP 5 tahap, anatomi DSN, serta lapisan abstraksi PDO.
- **Konfigurasi Lingkungan Multi-Driver:** Mengelola parameter koneksi pada .env dan config/database.php (SQLite/MySQL), memahami Lazy/Singleton Connection, serta memverifikasi status koneksi dengan php artisan db:show.
- **Mitigasi Mutlak SQL Injection:** Memahami mekanisme protokol dua fase (Prepare & Execute) pada PDO Parameter Binding yang memisahkan struktur perintah SQL dari nilai input pengguna.
- **Tiga Pendekatan Query di Laravel 11:** Mengimplementasikan Raw SQL (DB::select), Fluent Query Builder (DB::table) untuk agregasi laporan cepat, serta Eloquent ORM (Model) untuk logika bisnis.
- **Optimasi Kueri Relasi & Filter Dinamis:** Menerapkan perhitungan relasi efisien dengan withCount(), pencarian kondisional dengan when(), serta paginasi dengan withQueryString().
- **Transaksi Basis Data ACID & Concurrency Locking:** Membungkus operasi multi-tulis dalam DB::transaction(), menerapkan Pessimistic Locking (lockForUpdate()) untuk mencegah Race Condition kuota/stok, dan menangkap QueryException.
- **Komparasi HTTP Cookies vs Server Sessions:** Membedakan penyimpanan sisi klien (Cookies) dan sisi server (Sessions: File/Database/Redis) beserta atribut keamanan HttpOnly, Secure, dan SameSite.

- **Siklus Hidup Sesi & Autentikasi Aman:** Mengelola state sesi (Create, Read, Flash, Destroy), membangun sistem Login Auth::attempt(), regenerasi ID sesi (session()->regenerate()) anti-Session Fixation, serta proteksi rute dengan middleware auth.

II. Alat dan Bahan (Prerequisites)

Perangkat / Modul	Versi & Spesifikasi	Peran dalam Praktikum Minggu 6
Basis Kode Minggu 5	Proyek sim-del & SIPUS-Del	Melanjutkan proyek terbimbing (sim-del) dan proyek tugas mandiri (SIPUS-Del) dari Minggu 5.
PHP & Ekstensi PDO	PHP 8.2 atau 8.3 CLI	Ekstensi wajib aktif: pdo, pdo_mysql, pdo_sqlite, mbstring, openssl.
Mesin Basis Data	SQLite 3 / MySQL 8.0 (InnoDB)	Penyimpanan persisten relasional dengan dukungan penuh transaksi ACID & Row-Level Locking.
Database Client GUI	DBeaver / TablePlus / HeidiSQL	Inspeksi visual struktur tabel, foreign key, composite unique, dan tabel sessions.
Browser DevTools	Chrome / Edge Developer Tools	Menginspeksi header Set-Cookie serta atribut keamanan HttpOnly dan SameSite pada tab Application.
Version Control	Git CLI & Akun GitHub	Manajemen branch week6-db-session, Conventional Commits, dan pengumpulan repositori.

III. Landasan Teori dan Penjelasan Mendalam

3.1 Arsitektur Konektivitas Basis Data, Siklus Koneksi, & Lapisan PDO

Variabel PHP di dalam memori RAM bersifat efemeral (langsung musnah saat HTTP Response selesai dikirim). Untuk menyimpan data secara permanen, aplikasi Laravel bertindak sebagai Database Client yang berkomunikasi dengan server RDBMS melalui socket TCP/IP (Port 3306 untuk MySQL) atau berkas lokal SQLite dalam 5 tahapan siklus hidup koneksi:

- **1. Connection Open (TCP Handshake):** Membuka kanal socket jaringan menuju DB_HOST dan DB_PORT.
- **2. Authentication (Singleton 1x):** Memverifikasi DB_USERNAME dan DB_PASSWORD. Laravel menyimpan koneksi sebagai singleton sehingga autentikasi cukup dilakukan 1 kali per HTTP request.
- **3. Query Execution (Prepared Statement):** Mengirimkan template SQL terkompilasi (?) beserta parameter terikat melalui PDO untuk mencegah SQL Injection.
- **4. Result Fetching (Hydration):** Mengambil baris hasil query ke dalam memori PHP sebagai stdClass (Query Builder) atau objek Model (Eloquent ORM).
- **5. Connection Teardown:** Menutup koneksi secara otomatis pada saat siklus HTTP request selesai.

3.2 Transaksi Basis Data (ACID) & Pessimistic Locking (lockForUpdate)

Operasi bisnis kritis yang memodifikasi lebih dari satu tabel—seperti mengurangi sisa kuota mata kuliah lalu mencatat pengambilan KRS, atau mengurangi stok buku perpustakaan lalu mencatat transaksi peminjaman—wajib memenuhi prinsip ACID (Atomicity, Consistency, Isolation, Durability).

> **PERINGATAN KRITIS — BAHAYA RACE CONDITION & KEWAJIBAN LOCKFORUPDATE():** Apabila sisa kuota kelas atau stok buku tinggal 1, dan dua pengguna menekan tombol pada milidetik yang sama tanpa penguncian, keduanya dapat lolos pemeriksaan stok > 0 sehingga stok menjadi minus (-1). Bungkus seluruh proses di dalam DB::transaction() dan kunci baris menggunakan lockForUpdate() (SELECT ... FOR UPDATE) agar kueri kedua mengantre hingga transaksi pertama tuntas!

3.3 Manajemen Sesi Pengguna (HTTP Cookies vs Server-Side Sessions)

Protokol HTTP bersifat stateless. Untuk mengenali pengguna yang sedang login lintas halaman, Laravel mengirimkan Session ID terenkripsi ke peramban klien melalui header Set-Cookie (laravel_session), sementara payload data sesi disimpan secara aman di sisi server (tabel database sessions). Cookie sesi wajib dilindungi flag HttpOnly (mencegah pencurian cookie via JavaScript XSS) dan SameSite=Lax (mitigasi CSRF), serta wajib diregenerasi saat login (session()->regenerate()) guna menangkal serangan Session Fixation.

IV. Kegiatan Praktikum Terbimbing (Hands-on Labs)

LAB 1: Konfigurasi Konektivitas Database & Skema Migrasi KRS (35 Menit)

Buka proyek sim-del dari Minggu 5, verifikasi koneksi database, dan buat Model beserta Migration untuk Course dan Enrollment:

Snippet Kode: Verifikasi Koneksi & Pembuatan Model/Migration KRS

```
# 1. Verifikasi koneksi aktif dan daftar tabel database
php artisan db:show

# 2. Buat Model Course, Enrollment, dan CourseSeeder
php artisan make:model Course -m
php artisan make:model Enrollment -m
php artisan make:seeder CourseSeeder
```

Lengkapi skema migrasi tabel courses dan enrollments (lengkap dengan composite unique constraint):

Snippet Kode: Skema Migrasi courses dan enrollments

```
// database/migrations/xxxx_create_courses_table.php
Schema::create('courses', function (Blueprint $table) {
    $table->id();
    $table->string('code', 10)->unique();
    $table->string('name', 120);
    $table->unsignedTinyInteger('credits');
    $table->unsignedTinyInteger('semester');
    $table->unsignedInteger('quota')->default(40);
    $table->string('lecturer', 120);
    $table->timestamps();
});

// database/migrations/xxxx_create_enrollments_table.php
Schema::create('enrollments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('course_id')->constrained('courses')->cascadeOnDelete();
    $table->string('student_nim', 15);
    $table->string('student_name', 150);
    $table->string('academic_year', 25)->default('2025/2026-Genap');
    $table->timestamps();
    $table->unique(['course_id', 'student_nim']); // Cegah pendaftaran ganda!
});
```

Definisikan properti $fillable dan relasi One-to-Many pada app/Models/Course.php dan app/Models/Enrollment.php:

Snippet Kode: Model Eloquent Course.php dan Enrollment.php

```
// app/Models/Course.php
class Course extends Model {
    protected $fillable = ['code', 'name', 'credits', 'semester', 'quota', 'lecturer'];
    public function enrollments(): HasMany {
        return $this->hasMany(Enrollment::class);
    }
}

// app/Models/Enrollment.php
class Enrollment extends Model {
    protected $fillable = ['course_id', 'student_nim', 'student_name', 'academic_year'];
    public function course(): BelongsTo {
        return $this->belongsTo(Course::class);
    }
}
```

LAB 2: Fluent Query Builder, Eloquent withCount, & Transaksi ACID KRS (50 Menit)

Buat CourseController yang menerapkan pencarian dinamis, rekapitulasi Query Builder (DB::table), serta transaksi ACID Ambil & Batal KRS:

Snippet Kode: Implementasi CourseController.php (Query Builder & DB::transaction)

```
public function index(Request $request) {
    if ($request->filled('search')) {
        session(['last_krs_search' => $request->search]);
    }
    $courses = Course::withCount('enrollments')
        ->when($request->search, fn($q, $s) => $q->where('code', 'like', "%{$s}%")->orWhere('name', 'like',
"%{$s}%"))
        ->orderBy('code')->paginate(10)->withQueryString();

    $stats = [
        'total_courses'     => DB::table('courses')->count(),
        'total_quota_left'  => DB::table('courses')->sum('quota'),
        'total_enrollments' => DB::table('enrollments')->count(),
        'total_sks_taken'   => DB::table('enrollments')
            ->join('courses', 'enrollments.course_id', '=', 'courses.id')->sum('courses.credits'),
    ];
    return view('courses.index', compact('courses', 'stats'));
}

public function enroll(Request $request, int $id) {
    $validated = $request->validate([
        'student_nim'  => 'required|string|min:6|max:15',
        'student_name' => 'required|string|min:3|max:150',
    ]);
    try {
        DB::transaction(function () use ($validated, $id) {
            $course = Course::where('id', $id)->lockForUpdate()->firstOrFail();
            if ($course->quota < 1) {
                throw new \RuntimeException("Kuota kelas {$course->code} telah penuh!");
            }
            $course->decrement('quota');
            $course->enrollments()->create([
                'student_nim'   => strtoupper(trim($validated['student_nim'])),
                'student_name'  => $validated['student_name'],
                'academic_year' => '2025/2026-Genap',
            ]);
        }, 3);
        return back()->with('success', 'Berhasil mengambil KRS secara atomik!');
    } catch (\RuntimeException $e) {
        return back()->withInput()->with('error', $e->getMessage());
    } catch (QueryException $e) {
        Log::error('[DB EXCEPTION]: ' . $e->getMessage());
        return back()->withInput()->with('error', 'NIM sudah terdaftar pada mata kuliah ini.');
    }
}
```

LAB 3 & 4: Manajemen Sesi, Cookie, Autentikasi, & Audit DevTools (85 Menit)

Implementasikan AuthController untuk mengelola siklus hidup sesi autentikasi (Login, Regenerasi ID Sesi, Cookie, & Logout) serta proteksi middleware auth:

Snippet Kode: Implementasi AuthController.php & Proteksi Rute

```
public function login(Request $request) {
    $credentials = $request->validate(['email' => 'required|email', 'password' => 'required|string']);
    if (Auth::attempt($credentials, $request->boolean('remember'))) {
        $request->session()->regenerate(); // WAJIB: Cegah Session Fixation!
        session(['login_timestamp' => now()->format('d M Y, H:i:s') . ' WIB']);
        Cookie::queue('del_last_user_email', $request->email, 10080);
        return redirect()->intended(route('courses.index'))->with('success', 'Sesi Anda telah aktif.');
    }
    return back()->with('error', 'Kredensial login tidak valid.');
}

public function logout(Request $request) {
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();
    return redirect()->route('login')->with('success', 'Sesi server telah dihapus dengan aman.');
}
```

V. Tugas Mandiri Mahasiswa (Tugas Lanjutan dari Minggu 5)

> **JUDUL PENUGASAN MANDIRI LANJUTAN (SIPUS-DEL TAHAP 2):** Pengembangan Lanjutan Sistem Informasi Perpustakaan Kampus Del (SIPUS-Del): Implementasi Konektivitas Basis Data, Transaksi Sirkulasi Peminjaman & Pengembalian Buku Berbasis ACID (DB::transaction & lockForUpdate), Rekapitulasi Query Builder, serta Manajemen Sesi & Autentikasi Pustakawan pada Laravel 11.x

Spesifikasi Teknis Wajib (Melanjutkan Proyek SIPUS-Del Minggu 5)

- **1. Skema Tabel Transaksi Peminjaman (peminjamans) & Relasi Eloquent (Bobot: 20%):** Melanjutkan proyek SIPUS-Del Minggu 5 (tabel kategoris dan bukus). Buat migration tabel baru peminjamans dengan kolom: id, kode_pinjam (unik), buku_id (foreign key constrained ke bukus dengan cascadeOnDelete), nim_peminjam, nama_peminjam, tanggal_pinjam, tanggal_jatuh_tempo, tanggal_kembali (nullable), status ('dipinjam' / 'dikembalikan'), dan timestamps. Definisikan Model Peminjaman ($fillable, belongsTo Buku) dan relasi hasMany(Peminjaman::class) pada Model Buku.
- **2. Transaksi Basis Data ACID (DB::transaction) & Concurrency Locking (Bobot: 25%):** Fitur Peminjaman Buku wajib dibungkus DB::transaction() dengan Buku::where('id', $id)->lockForUpdate()->firstOrFail(), memeriksa stok > 0 (tolak & rollback jika stok = 0), mencegah NIM yang sama meminjam buku yang sama yang belum dikembalikan, mengurangi stok ($buku->decrement('stok')), dan mencatat ke tabel peminjamans. Fitur Pengembalian Buku wajib menggunakan DB::transaction() + lockForUpdate() untuk mengubah status menjadi 'dikembalikan', mengisi tanggal_kembali, dan mengembalikan stok buku ($buku->increment('stok')). Tangkap QueryException dan catat ke Log::error().
- **3. Fluent Query Builder (DB::table), Agregasi withCount, & Filter Dinamis (Bobot: 15%):** Perbarui katalog buku agar menampilkan jumlah peminjaman menggunakan Buku::with('kategori')->withCount('peminjamans'), filter pencarian dinamis when() (judul/penulis/ISBN & kategori), serta paginasi withQueryString(). Tampilkan kartu statistik perpustakaan menggunakan DB::table() (Total Judul Buku, Total Stok Tersedia sum('stok'), Total Buku Sedang Dipinjam, dan Total Dikembalikan).
- **4. Manajemen Sesi Pengguna (Session & Cookie) & Autentikasi Pustakawan (Bobot: 25%):** Implementasikan AuthController untuk Login dan Logout Pustakawan menggunakan Auth::attempt(), wajib

menjalankan $request->session()->regenerate() saat login dan invalidate() + regenerateToken() saat logout. Proteksi seluruh rute mutasi (CRUD buku/kategori, peminjaman, pengembalian) dengan middleware auth. Simpan state sesi pengguna (misal histori pencarian/peminjam terakhir) di Session dan minimal 1 HTTP Cookie menggunakan Cookie::queue().

- **5. Seeder, Bukti Pengujian Transaksi ACID, & Audit Cookie DevTools (Bobot: 15%):** Sediakan seeder akun Pustakawan uji (pustakawan@del.ac.id / password123), sampel buku dengan stok = 0 dan stok = 1, serta sampel transaksi peminjaman. Lampirkan screenshot bukti peminjaman/pengembalian, penolakan saat stok = 0, dan inspeksi Cookie HttpOnly pada Browser DevTools di README.md.

VI. Instruksi Git & Pengumpulan Tugas

1. Lanjutkan repositori proyek SIPUS-Del Minggu 5 dan gunakan konvensi pesan commit standar (Conventional Commits):

Snippet Kode: Konvensi Commit Git Minggu 6

```
git checkout -b week6-db-session
git commit -m "feat(migration): create peminjamans table with foreign key to bukus"
git commit -m "feat(transaction): implement ACID book borrowing and return with lockForUpdate"
git commit -m "feat(query-builder): add library analytics dashboard and dynamic search"
git commit -m "feat(auth-session): implement librarian login, session regeneration, and cookies"
```

2. DILARANG KERAS meng-commit berkas .env atau folder vendor/ ke GitHub. Pastikan kredensial akun uji Pustakawan tertulis jelas di README.md.

3. Format Nama Repositori Publik GitHub: ppw-2026-week6-[NIM] (atau branch week6-db-session pada repositori Minggu 5).

VII. Rubrik Penilaian Praktikum Analitik

Kriteria Evaluasi	Bobot	Sangat Baik (85-100)	Baik (70-84)	Kurang (<70)
1. Konektivitas DB & Migrasi peminjamans	**20%**	Konfigurasi DB bersih; migrasi peminjamans memiliki FK cascadeOnDelete presisi; relasi Buku <-> Peminjaman sempurna.	Migrasi dan relasi model Buku dan Peminjaman berjalan baik sesuai spesifikasi.	Tabel peminjamans belum memiliki foreign key constraint atau relasi Eloquent gagal.
2. Transaksi ACID & Concurrency Locking	**25%**	Pinjam & kembali buku dibungkus DB::transaction() + lockForUpdate(); mutasi stok atomik; stok 0 & QueryException tertangani.	DB::transaction() diterapkan pada sirkulasi buku; stok berkurang dan bertambah dengan benar.	Mutasi stok tidak memakai DB::transaction() / lockForUpdate() atau stok menjadi minus.
3. Query Builder, withCount, & Filter	**15%**	Statistik DB::table() akurat; katalog memakai withCount('peminjamans') bebas N+1; pencarian when() & paginasi mulus.	Statistik Query Builder dan filter pencarian berfungsi dengan baik; paginasi aktif.	Tidak menggunakan Query Builder atau masih terjadi N+1 query pada daftar buku/peminjaman.

Kriteria Evaluasi	Bobot	Sangat Baik (85-100)	Baik (70-84)	Kurang (<70)
4. Manajemen Sesi, Cookie, & Auth	**25%**	Login/Logout aman dengan session()->regenerate() & invalidate(); rute mutasi diproteksi auth; Session state & Cookie aktif.	Login/Logout dan proteksi middleware auth berfungsi baik; Flash session tampil rapi.	Belum ada regenerasi sesi, rute transaksi belum diproteksi auth, atau tanpa manajemen sesi.
5. Seeder, Bukti Uji ACID, & Dokumentasi	**15%**	Seeder akun pustakawan & peminjaman lengkap; README memuat ERD, bukti uji rollback stok 0, dan audit Cookie HttpOnly.	Seeder berfungsi; dokumentasi README lengkap dengan panduan akun login dan screenshot.	Seeder gagal; .env bocor ke GitHub; README tidak memuat bukti pengujian transaksi.